

Lean 4 Summary Sheet

1. Term Builders for Prop

These are constructors used to explicitly build terms inhabiting Prop in Lean4. The number of underscores following the constructor indicates the number of terms that can be passed to it. Note it can be useful, when using tactics, to only partially apply some constructors.

1. `And.intro` _ _

e.g. Given terms $p : P$ and $q : Q$ the term `And.intro p q` : $P \wedge Q$.

This can also be written $\langle p, q \rangle : P \wedge Q$

2. `And.left` _

e.g. Given a term $t : P \wedge Q$, the term `And.left t` : P

This can also be written $t.left$: P

3. `And.right` _

e.g. Given a term $t : P \wedge Q$, the term `And.right t` : Q

This can also be written $t.right$: Q

4. `Or.intro_left` _ _

e.g. Given a term $p : P$ and a Prop Q the term `Or.intro_left Q p` : $P \vee Q$

5. `Or.intro_right` _ _

e.g. Given a term $p : P$ and a Prop Q the term `Or.intro_right Q p` : $Q \vee P$

6. `Or.elim` _ _ _

e.g. Given terms $t : P \vee Q$, $f : P \rightarrow R$, and $g : Q \rightarrow R$ the term `Or.elim t f g` : R

7. `fun` _ $\Rightarrow$ _

e.g. Given an identifier (any lower case english string) say $t : P$ and a term $e : Q$ we can construct `fun t : P => e` which has type $P \rightarrow Q$

This can be written without the explicit typing on the variable t . It can also be written with a λ instead of `fun`

$\lambda t \Rightarrow e$ is a term of type $P \rightarrow Q$. Lean can infer the type of t .

Notice the use of $\Rightarrow$ instead of the $.$ used in our on-paper lambda calculus.

2. Unicode Symbols in VSCode

Type a backslash `\` followed by the keyword to insert the symbol:

Name	Lean Input	Symbol	Meaning
And	<code>\and</code>	$\wedge$	Logical AND
Or	<code>\or</code>	$\vee$	Logical OR
Not	<code>\not</code>	$\neg$	Logical NOT
Implies	<code>\to</code>	$\rightarrow$	Implication
Forall	<code>\forall</code>	$\forall$	Universal quantifier
Exists	<code>\exists</code>	$\exists$	Existential quantifier
Natural numbers	<code>\mathbb{N}</code>	$\mathbb{N}$	Set of naturals
Alpha	<code>\alpha</code>	α	Greek letter alpha
Beta	<code>\beta</code>	β	Greek letter beta
Turnstile	<code>\vdash</code>	$\vdash$	Provability symbol
Lambda	<code>\lambda</code>	λ	Lambda abstraction
Left Angle bracket	<code>\langle</code>	$\langle$	AND introduction
Right Angle bracket	<code>\rangle</code>	$\rangle$	AND introduction

3. Keywords

Avoid using these as identifiers (names) for terms, types, or variables.

- **variable**

This is used to declare variables inhabiting particular types.

- **theorem**

This is used to state a theorem in Lean. It has the following pattern:

`theorem <name> <list of hypotheses> : <goal-type> := <proof-term>`

- **example**

This is used to state an anonymous theorem in Lean. It has the following pattern:

`example <list of hypotheses> : <goal-type> := <proof-term>`

- **def**

- **with**

- **match**

4. Tactics for Proofs in Prop

Rather than writing proof-terms explicitly, these tactics help authoring proofs about propositions. If we think of the term constructors (equiv rules of inference) as the fundamental rules of reasoning, then tactics can be thought of as higher-level, meta, reasoning tools. These metaprograms are instructions for creating proof-terms.

One can open tactic mode by writing “by” at the start of a proof. One then applies a list of tactics in order to prove the goal. These can be written one per line, or separated by semi-colon on the same line.

- **intro** _ _ _ ...

This mimics the use of the deduction theorem. If the goal is an implication, then it uses the name(s) passed to intro to put a term of the antecedent type in the current context. If there are nested implications, then intro can be passed a number of identifiers to move all the antecedents to the current context in one line.

e.g. If the current goal is $A \rightarrow (B \rightarrow (C \wedge D \rightarrow E))$, then we can apply `intro a b t` which will update the goal to E and put three terms $a : A$, $b : B$, and $t : C \wedge D$ into the context.

It takes care of all the implication introductions behind the scenes.

- **apply** _

If our goal is C and we have a theorem “robertsTheorem” which proves C based on three hypotheses of type P, Q, R , then “apply robertsTheorem” will change the proof-state to have three goals corresponding to the three hypotheses P, Q, R .

We say, in order to prove C it is sufficient, by robertsTheorem, to prove P, Q, R separately.

Using this in Prop can split a proof of conjunction $P \wedge Q$ to two proofs; one of P and another of Q . We can write “apply And.intro” in order to achieve this.

Similarly, one can split a proof into two cases using “apply Or.elim t” where $t : P \vee Q$. If the original goal was C , then this will split the proof into two proofs: one with goal $P \rightarrow C$, the other with $Q \rightarrow C$.

- **exact** _

e.g. If the goal of the current proof-state is P and $t : P$, then `exact t` completes the proof.

- **have** _ := _

e.g. This tactic introduces an intermediate term into the context. It gives a name to a subproof used on the way to the goal. It doesn’t change the goal of the proof-state. If $e : P$, then `have t := e` introduces the term t of type P into the context.

If the proof is particularly long, then using have can break the proof up into manageable steps.

Tactics for Proofs on Peano Arithmetic

Each of the tactics in the previous section remain important when writing proofs about any area of mathematics. On top of those are the following new tactics that you will find useful when proving theorems about the natural numbers.

- `rfl`

If the goal can be closed by computing two sides of an equation down to the same value, then `rfl` will close the goal. That is, `rfl` will determine whether two expressions are equal up to computation. Such proofs are said to be “mere computation”.

- `rw [_, _, _, ...]`

Interesting theorems require more than mere computation to prove them. In this case, one can make progress proving an equation by manually calling rewrites by other theorems and hypothesis in the local context. The idea here is that in order to prove an equation, we can apply previous results to terms in the equation in order to get it down to a trivial statement like $a = a$ that `Eq.refl` can close.

This tactic will take a comma separated list of theorems and apply them in succession. See the induction section below for an example.

- `simp`

This tactic is similar to `rfl` in that it applies equational theorems to the goal. However, the database of theorems and lemmas that `simp` has access to can be added to be the programmer. The `@[simp]` attribute tag can be added to a theorem to add it to the `simp` tactic. Therefore `simp` is, in a sense, a more powerful `rfl` as it can rewrite with theorems and lemma, not just definitions.

- `induction _ with _`

This is one of the key tactics when working with inductive types like the natural numbers. Here is an example of the pattern used on a theorem about natural numbers.

```
theorem zero_add : ∀ x : ℕ, zero + x = x :=
  by
    intro a
    induction a with
    | zero      => rfl
    | succ k ih => rw [add_succ, ih]
```

This requires the proof of the base cases i.e. the case $a = \text{zero}$ and the induction step where $a = \text{succ } k$ for some natural number k . In the induction step we have access to the proof of the theorem at $a = k$; here we have called that proof `ih` - induction hypothesis.

– `obtain <_,> := _`

If we want to unpack hypotheses defined by inductive types like $\exists x, P\ x$ or $P \wedge Q$, then we can use the `obtain` tactic. If $t : P \wedge Q$, then `obtain <p, q> := t` returns the proof $p : P$ and the proof $q : Q$ from inside the conjunction.

Similarly, the witness and proof inside $t : \exists x : \alpha, P\ x$ can be extracted using the `obtain` tactic. For example, `obtain <a, pa> := t` takes the proof t from the context and returns a term $a : \alpha$ and the proof pa that a satisfies the predicate P .

– `calc`

The `calc` tactic provides a different way to present a proof using a succession of `rw` rewrites. Here is the same proof of `zero_add` above, but this time presented using the `calc` tactic.

```
theorem zero_add :  $\forall x : \mathbb{N}, \text{zero} + x = x$  :=  
  by  
    intro a  
    induction a with  
    | zero      => rfl  
    | succ k ih =>  
      calc  
        zero + succ k  
        = succ (zero + k) := by rw [PA4]  
        _ = succ k       := by rw [ih]
```

This allows for the presentation of the calculation in a manner closer to how we might write it on paper. In this way the proof is clearer to the reader, but more cumbersome to write. Note that one can use more than one `rw` on a single line.